Contents

Contents **i**

List of Figures **ii**

List of Tables **iii**

I Introduction **1**

II What is TAMPINES **2**

IIISteam Tables **3**

IVRankine Cycle Derivations **4**

1 Mass, Energy and Momentum Balance **6**

1.1 Raw forms of mass, momentum and energy balance 6

1.2 converting forms to mass flowrate 6

1.3 Approach to Solving nonlinear Equations 7

Bibliography **10**

List of Figures

List of Tables

Acknowledgments

Part I

Introduction

Part II

What is TAMPINES

Part III

Steam Tables

Part IV

Rankine Cycle Derivations

In this part, we consider a four control volume system representing a simple Rankine cycle. This will form the theoretical basis to

Chapter 1

Mass, Energy and Momentum Balance

In OpenFOAM, the PIMPLE algorithm is often used to solve for incompressible fluid flow [Devolder et al., 2015], this is one of the many standard solvers for solving flow in OpenFOAM [Penttinen, Yasari, and Nilsson, 2011]. Jasak has published this in his PhD thesis, available online, [Jasak, 1996] on page 143-149.

1.1 Raw forms of mass, momentum and energy balance

Let's consider mass balance and momentum balance:

$$\frac{\partial \dot{m}}{\partial t} = \sum_i m_i \tag{1.1}$$

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.2}$$

$\nabla \cdot \tau$ represents shear, ρg represents the gravity body force, $-\nabla P$ represents the pressure gradient.

$$V_j \frac{\partial \rho_i h_i}{\partial t} + \sum_j \dot{m}_j h_j = Q_i - W_i + \sum_j \tau_j \cdot \mathbf{u}_j \tag{1.3}$$

Where $+\sum_j \tau_j \cdot \mathbf{u}_j$ is basically the friction loss terms where energy is transformed from kinetic energy to sensible heat. Q and W represent the thermodynamic work

1.2 converting forms to mass flowrate

It is often more convenient to convert the momentum equations to be in terms of mass flowrates. This is for 1D case.

To do so, we transform this by consider this 3D equation.

$$\frac{\partial(\rho\mathbf{u})}{\partial t} + \nabla \cdot (\rho\mathbf{u}\mathbf{u}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.4}$$

We need to consider just the x component, for a simple 1D case.

Then multiply by the cross sectional area.

1.3 Approach to Solving nonlinear Equations

Jasak in his PhD thesis outlines the methodology for solving this set of equation [Jasak, 1996] on page 143-149. These were the foundations for what would eventually become OpenFOAM Jasak, Jemcov, Tukovic, et al., 2007 licensed under GPLv3.

The momentum equation is quadratic in u, which makes the equation solution nonlinear. Solving this nonlinear equation would either need a nonlinear solver such as Newton Raphson method, or solving a linearised form of these equations, and iterating to convergence. The latter was chosen [Jasak, 1996].

$$\nabla \bullet (\mathbf{u}\mathbf{u}) = \sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f = \sum_f F\mathbf{u}_f = a_p\mathbf{u}_p + \sum_N a_N\mathbf{u}_n \tag{1.5}$$

Finite Volume Discretisaion, Face and Centroid Properties

Section 3.3.1 within Jasak's thesis explains these terms.

First, we need to understand that Gauss's theorem is used in Finite Volume analysis, which is the bedrock of OpenFOAM solvers. $\sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f$ represent the surface vector $\mathbf{S}$ and this will have a dot product with the velocity at the faces of the control volume $\mathbf{u}_f$.

Note that the subscript f represents values at the faces.

$\sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f$ can be written in shorthand $\sum_f F\mathbf{u}_f$.

Now, we may not have the values of the velocities or fluxes at the faces. So we have to estimate it based on the properties at the centroid of the cell denoted p, and those of the neighbours denoted n.

Thus, the surface values denoted f are estimated based on the values at the cell centroids p and n based on some kind of interpolation scheme. This can be linear interpolation, or some upwind differencing scheme. This is described in page 81 of Jasak's thesis [Jasak, 1996].

With linear interpolation at the face, the solution can become unbounded. With upwind differencing, the solution is bounded, but diffusive. Much of the discussion revolves around how to estimate properties at the faces. This is why a whole host of differencing schemes comes up. Eg. at page 110 of Jasak's thesis [Jasak, 1996].

Linearisation and Predictor values

Given a very nonlinear system of equations. It was desirable to consider solving a linearised version of the momentum equations. We want to solve for the velocity $\mathbf{u}^{t+\Delta t}$ at the next timestep. But we only know $\mathbf{u}^t$.

We know that the face fluxes should be based on values in the cell centroids for both the cell p and that of the neighbouring cell n, hence we use:

$$a_p \mathbf{u}_p + \sum_N a_N \mathbf{u}_n$$

However, we have $\mathbf{uu}$ terms. To solve this problem and linearise this, we can use $\mathbf{u}^t \mathbf{u}^{t+\Delta t}$. Basically, $\mathbf{u}^t$ is known. This is the velocity at the previous timestep, and is therefore a constant. This will then help form the respective a_p and a_n constants so that the equations are linear.

Let's go back to our equations:

$$\frac{\partial(\rho \mathbf{u})}{\partial t} + \nabla \cdot (\rho \mathbf{uu}) = -\nabla P + \nabla \cdot \tau + \rho g \tag{1.6}$$

In the incompressible assumption,

$$\frac{\partial(\mathbf{u})}{\partial t} + \nabla \cdot (\mathbf{uu}) = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.7}$$

We can do time discretisation:

$$\frac{\mathbf{u}^{t+\Delta t} - \mathbf{u}^t}{\Delta t} + \nabla \cdot (\mathbf{uu}) = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.8}$$

Now, remember, based on divergence theorem we need fluxes or properties at the faces:

$$\frac{\mathbf{u}^{t+\Delta t} - \mathbf{u}^t}{\Delta t} + \sum_f \mathbf{S} \cdot \mathbf{u}_f \mathbf{u}_f = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.9}$$

And since we cannot find properties at the faces, we have to estimate them using properties at the centroids.

Just to be clear, let's label the velocities at their proper locations, and then linearise the equation using some semi implicit time discretisation:

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S} \cdot \mathbf{u}_f^t \mathbf{u}_f^{t+\Delta t} = -\nabla P/\rho + \nabla \cdot \tau/\rho + g \tag{1.10}$$

The fluxes at the faces can be further estimated using some interpolation scheme. The easiest to understand is a linear scheme. Herein, we use some weighting factor to estimate the flux at the face between the cell and the other cell.

$$\mathbf{u}_f^t = w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t$$

Substituting this back:

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S}\cdot(w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t)(w\mathbf{u}_p^{t+\Delta t} + (1-w)\mathbf{u}_n^{t+\Delta t}) = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \quad (1.11)$$

To make life easier for us, we can choose not to even bother with explicitly solving for the velocities of the neigbours

$$\frac{\mathbf{u}_p^{t+\Delta t} - \mathbf{u}_p^t}{\Delta t} + \sum_f \mathbf{S}\cdot(w\mathbf{u}_p^t + (1-w)\mathbf{u}_n^t)(w\mathbf{u}_p^{t+\Delta t} + (1-w)\mathbf{u}_n^t) = -\nabla P/\rho + \nabla\cdot\tau/\rho + g \quad (1.12)$$

Messy as the equation is, we now have a linear equation in velocities for each centroid at the next timestep $u_p^{t+\Delta t}$ and $u_n^{t+\Delta t}$.

We can then simplify this equation as such:

$$a_p\mathbf{u}_p^{t+\Delta t} = \mathbf{H}(\mathbf{u}^t) - \nabla P/\rho + \nabla\cdot\tau/\rho + g \quad (1.13)$$

$$\mathbf{u}_p^{t+\Delta t} = \frac{\mathbf{H}(\mathbf{u}^t)}{a_p} - \frac{\nabla P/\rho}{a_p} + \frac{\nabla\cdot\tau/\rho}{a_p} + g/a_p \quad (1.14)$$

Of course, the estimates for $\mathbf{u}^{t+\Delta t}$ are not going to be accurate. They are initial guess or **predicted** values.

We need to do some further correction to these predicted values in order to arrive at the correct answer.

Corrector steps

So far, we've only bothered with the momentum equation and ignored mass balance (or continuity in this case). Thus, there is no guarantee that the

Bibliography

Devolder, Brecht et al. (2015). "A review of the implicit motion solver algorithm in OpenFOAM® to simulate a heaving buoy". In: *18th Numerical Towing Tank Symposium (NuTTS'15)*.

Penttinen, Olle, Ehsan Yasari, and Håkan Nilsson (2011). "A pimplefoam tutorial for channel flow, with respect to different LES models". In: *Practice Periodical on Structural Design and Construction* 23.2, pp. 1–23.

Jasak, Hrvoje (1996). "Error analysis and estimation in the Finite Volume method with applications to fluid flows". In.

Jasak, Hrvoje, Aleksandar Jemcov, Zeljko Tukovic, et al. (2007). "OpenFOAM: A C++ library for complex physics simulations". In: *International workshop on coupled methods in numerical dynamics*. Vol. 1000. Dubrovnik, Croatia), pp. 1–20.